Eniac TimeSheet Platform

Product & Feature Specification

1. Document Purpose

This document translates the implemented Eniac codebase into a detailed feature specification. It is intended to be usable by product managers, business analysts, designers, QA engineers, developers, implementation teams, and stakeholders who need to understand what the platform currently does, who can use each capability, how major workflows behave, and where the implementation contains limitations or unfinished areas.

Specification basis
This is a codebase-derived specification, not a future-state product requirements document. Features described as implemented are based on observable source-code behavior. Where the UI suggests a capability but the API is incomplete or inconsistent, the discrepancy is explicitly identified.

2. Product Overview

Eniac is a workforce/project operations platform designed to coordinate projects, project teams, supervisors, weekly employee timesheets, timesheet review, notifications, activity history, project documents, and administrative reporting.

2.1 Product goals represented by the implementation

- Provide administrators with centralized project and workforce administration.
- Allow employees to record weekly project work and submit timesheets.
- Allow supervisors/admins to review, approve, or decline submitted timesheets.
- Provide traceability through activities and user notifications.
- Store project-related documents alongside project information.
- Provide management reporting around hours, overtime, project allocation, and timesheet status.
- Separate the experience and permissions of administrators, ordinary users, and supervisors.

3. Personas & Roles

Role/persona	Primary responsibilities	Main feature areas
Administrator	Manage platform users, projects, teams, supervisors, timesheets and reports.	All admin modules; approvals; reporting; settings
Employee/User	Work on assigned projects, maintain weekly timesheets, submit work, view notifications and project information.	Dashboard; projects; timesheets; submissions; notifications; settings
Supervisor	Review work of assigned teams/projects and participate in approvals.	Supervisor timesheets; supervisor approvals; project supervision
System/Cron process	Generate deadline-related notifications based on project deadlines.	Deadline notifications

4. Feature Inventory

ID	Feature	Area	Priority	Status
F-001	Authentication & session management	Identity	P0	Implemented with limitations
F-002	Role-based access control	Security	P0	Implemented; policy inconsistencies exist
F-003	User management	Administration	P0	Implemented
F-004	Supervisor management	Administration	P0	Implemented
F-005	Project management	Projects	P0	Implemented
F-006	Project team management	Projects	P0	Implemented
F-007	Project document management	Projects	P1	Implemented; frontend upload contract needs alignment
F-008	Employee dashboard	User experience	P1	Implemented
F-009	Admin dashboard	Management	P1	Implemented

F-010	Weekly timesheet creation	Timesheets	P0	Implemented
F-011	Timesheet editing	Timesheets	P0	Implemented with lifecycle restrictions
F-012	Timesheet submission	Timesheets	P0	Implemented
F-013	Timesheet withdrawal	Timesheets	P1	Implemented
F-014	Timesheet approval/decline	Approvals	P0	Implemented
F-015	Notifications	Engagement	P1	Implemented; ownership hardening needed
F-016	Activity history/audit trail	Auditability	P1	Implemented; access hardening needed
F-017	Management reports	Reporting	P1	Implemented
F-018	User settings	Settings	P2	UI exists; persistence scope should be validated
F-019	Password recovery	Identity	P1	Not implemented (501)
F-020	Token refresh	Identity	P0	Service capability exists but route/client flow incomplete
F-021	Deadline notification automation	Automation	P1	Implemented endpoint; scheduler configuration external

5. Global UX & Navigation

The application is a React single-page application with route spaces organized around role.

Experience	Representative navigation
Admin	Dashboard, Projects, Users, Supervisors, Timesheets, Approvals, Reports, Settings
User	Dashboard, Projects, Timesheets, Submissions, Notifications, Settings
Supervisor	Timesheets, Approvals
Authentication	Admin login, User login

5.1 Common UI patterns

- Data tables for users, projects, timesheets, approvals and history.
- Status badges for project/timesheet state.
- Cards/stat tiles for dashboard summaries.
- Modal/dialog interactions for editing, review, confirmation and assignment operations.
- Loading and error states through reusable components.
- Toast notifications for user-facing mutation feedback.
- Responsive navigation/layout intended for desktop and smaller screens.

6. F-001 — Authentication & Session Management

Purpose: establish a verified application identity and maintain a session across protected API calls.

User journey

1. User opens the appropriate login screen.
2. User enters email and password.
3. Client calls POST /api/v1/auth/login.
4. Backend validates input and finds an active user by normalized email.
5. Password is verified using bcrypt.
6. Backend returns an access token and user profile and sets an HttpOnly refresh-token cookie.
7. Client stores the access token and loads protected application data.

Rules

- Email matching is normalized/lowercased.
- Inactive users cannot authenticate.
- Access token lifetime is approximately one hour.
- Refresh token lifetime is approximately seven days.
- Refresh tokens are stored server-side in the sessions collection.
- Logout deletes the server-side session and clears the cookie.

Current limitations

- A refresh route is not exposed even though the service layer contains refresh-session logic.
- Password forgot/reset endpoints explicitly return 501/not implemented.
- The frontend stores the access token in localStorage; XSS prevention and CSP hardening are therefore important.

API	Purpose	Expected outcome
POST /auth/login	Login	User + access token + refresh cookie
GET /auth/me	Session identity	Current authenticated user
POST /auth/logout	Logout	Session revoked/cookie cleared
POST /auth/forgot-password	Password recovery	Not implemented
POST /auth/reset-password	Password reset	Not implemented

7. F-002 — Role-Based & Resource-Based Access Control

Purpose: ensure that users can only perform actions appropriate to their role and relationships to resources.

Capability	Admin	Employee	Supervisor
Manage users	Yes	No	No
Manage supervisors	Yes	No	No
Create projects	Yes	No	No
Edit projects	Yes	No	Intended for assigned projects; current route middleware is stricter
Delete projects	Yes	No	No
View assigned projects	Yes	Yes	Yes
Create own timesheet	Yes/subject to service rules	Yes	Yes
Edit own editable timesheet	Yes	Yes	Yes
Review/approve timesheets	Yes	No	Yes, scoped by supervisor rules
View management reports	Yes	No	No
View notifications	Own	Own	Own

Authorization note
The implementation has multiple access-policy layers. Some policies disagree—for example, supervisor access to a timesheet can be granted by middleware but rejected by the controller. This should be normalized into one canonical authorization policy.

8. F-003 — User Management

Purpose: allow administrators to manage employee identities and account status.

Supported data

Field	Purpose
Name	Employee display name
Email	Login/communication identifier

Employee ID	Internal employee identifier; unique
Department	Organizational classification
Role	admin or user
Supervisor flag	Determines whether a user can act as a supervisor
Supervisor ID	Links employee to a supervisor
Status	active/inactive
Password	Stored as bcrypt hash; never returned in user DTO

Features

- List users.
- View a specific user.
- Create user.
- Update user.
- Deactivate user.
- Activate user.
- Assign or remove supervisor capability.
- Assign a user's supervisor.

Business rules

- Email and employee ID are unique at the database level.
- Only administrators can manage user accounts.
- Password changes are handled separately from ordinary user DTO responses.

API	Function
GET /users	List users
GET /users/:id	View user
POST /users	Create
PATCH /users/:id	Update
POST /users/:id/deactivate	Deactivate
POST /users/:id/activate	Activate
PATCH /users/:id/supervisor	Assign/remove supervisor capability
GET /supervisors	List active supervisors
GET /supervisors/:id/users	List users supervised by a supervisor

9. F-004 — Supervisor Management

Purpose: model supervisory relationships and enable supervisors to participate in work review.

- Admin can assign a user as a supervisor.
- Admin can remove supervisor capability.
- Users can have a supervisorId relationship.
- Projects can have an assigned supervisor.
- Supervisor-facing timesheet/approval screens are available only when isSupervisor is true.
- Supervisor review rules are intended to consider project assignment and supervised users.

Supervisor access concepts

```
Supervisor capability
        |
        +--> supervised users
        |
        +--> supervised projects
        |
        +--> timesheet review
        |
        +--> approval / decline
```

10. F-005 — Project Management

Purpose: provide a central work container for client/SOW information, dates, team members, management ownership and project documents.

Project fields

Field	Description
Name	Project name
SOW number	Statement-of-work identifier; unique
Client	Customer/client
Description	Project description
Start date	Project start
End date	Project end
Deadline	Operational deadline used for notifications
Status	draft, active, completed, overdue, archived
Manager	Assigned project manager/user
Supervisor	Assigned supervisor
Team members	Users assigned to the project
Documents	Project document references

Project operations

- Create project.
- View project.
- Update project.
- Delete project.
- Add team member.
- Remove team member.
- Assign/change supervisor.
- List projects according to user access.

Access model

- Administrators have broad access.
- Team members can access assigned projects.
- Assigned supervisors can access relevant projects.
- Project access is enforced by backend resource middleware/services, not only by frontend navigation.

Project statuses

Status	Typical meaning
Draft	Project is being prepared/configured.
Active	Project is currently operational.
Completed	Project work is completed.
Overdue	Deadline/end-date condition indicates overdue work.
Archived	Project retained for historical purposes but no longer operational.

11. F-006 — Project Team Management

Purpose: control which users are associated with a project and therefore which users can perform project-scoped work.

- Add a user to a project team.
- Remove a user from a project team.
- Assign a manager and supervisor.
- Use team membership as an input to project access and timesheet eligibility.

Product implication
Team membership is not merely informational. It participates in authorization, so removing a member can change what that user can see or submit against.

12. F-007 — Project Document Management

Purpose: associate files with projects and store them in Vercel Blob with metadata in MongoDB.

Supported workflow

8. User opens a project to which they have access.
9. User selects a file.
10. Backend validates MIME type and file size.
11. Backend sanitizes the original filename.
12. File is uploaded to Vercel Blob.
13. Document metadata is stored in MongoDB.
14. Activity record is generated.
15. Project document list displays the stored metadata/link.

Constraints

- Maximum file size: 10 MB.
- Supported formats include PDF, common image formats, Word, Excel, TXT and CSV.
- Blob access is currently configured as public.

Current implementation issue

- The backend expects multipart/form-data with a file, while the frontend document service currently exposes a JSON-style createDocument call. The client implementation should be aligned with the backend upload contract.
- Frontend and backend document DTOs also use different field names/types.
- Deletion authorization should explicitly define whether uploader, supervisor, project manager or admin can delete documents.

13. F-008 — Employee Dashboard

Purpose: provide the employee with a summary of work and pending actions.

- Project-oriented summary information.
- Timesheet information and recent activity.
- Notification/deadline awareness.
- Navigation into timesheets, submissions, projects and notifications.

Exact KPI calculations should be treated as UI implementation details and validated against the current page components before being used as contractual reporting definitions.

14. F-009 — Admin Dashboard

Purpose: provide management-level visibility across users, projects, timesheets, approvals and operational activity.

- Summary/stat cards.
- Project and timesheet status information.
- Deadline/activity information.
- Navigation to administrative management and reporting.
- Charts are implemented using Recharts in the reporting/dashboard area.

15. F-010 — Weekly Timesheet Creation

Purpose: let users record project work for a specific Monday-based week.

Core structure

Component	Behavior
Project	Timesheet belongs to one project.
Week start	Normalized to Monday.
Daily hours	Mon–Sun hours by entry.
Entry type	regular or overtime.
Description	Required when an entry contains hours.
Notes	Optional weekly notes.
Totals	Regular, overtime and total hours calculated server-side.
Status	Lifecycle state controls edit/submit/review behavior.

Validation rules

- Week start is normalized/validated as Monday.
- Regular work is Monday–Friday.
- Overtime is Saturday–Sunday.
- Individual day hours cannot exceed 24.
- Entries containing hours require a description.
- Totals are recalculated by the backend rather than trusted from the client.
- A user/project/week combination is unique.

Creation journey

16. User selects an accessible project.
17. User selects a week.
18. User enters daily hours and descriptions.
19. Client submits the timesheet.
20. Server validates the project relationship and week.
21. Server calculates totals and creates a draft.
22. Draft becomes editable until submitted.

16. F-011 — Timesheet Editing

Purpose: allow correction of a timesheet while it is in an editable state.

- Draft timesheets can be edited.
- Declined/withdrawn records can be corrected and resubmitted according to service rules.
- Pending/approved timesheets are protected from ordinary editing.
- Only authorized users can edit a timesheet.
- Totals are recalculated after updates.

Recommended UX behavior

- Clearly show why an edit is disabled.
- Show decline reason when a supervisor/admin has declined a submission.
- Show last submission/review state and timestamps.

17. F-012 — Timesheet Submission

Purpose: move an editable timesheet into the review workflow.

```
Editable
   |
   | Submit
   v
Pending
   |
   +--> Approved
   |
   +--> Declined --> editable
   |
   +--> Withdrawn --> editable
```

- Submission sets the review workflow in motion.
- The system records submission timing.
- Notification and activity side effects are generated.
- A submitted timesheet should no longer be freely editable while pending.

18. F-013 — Timesheet Withdrawal

Purpose: let an employee retract a pending submission before review.

- Withdrawal is available to the timesheet owner subject to state rules.
- Withdrawal changes the state to withdrawn.
- The record can subsequently be edited/resubmitted according to service rules.
- Activity/notification side effects support traceability.

19. F-014 — Timesheet Approval & Decline

Purpose: provide a controlled review process for submitted timesheets.

Reviewer capabilities

- View pending submissions within their authorization scope.
- Approve a pending timesheet.
- Decline a pending timesheet with a reason.
- Review history/activity around a timesheet.

Approval journey

23. Employee submits timesheet.
24. Timesheet becomes pending.
25. Eligible reviewer sees it in approvals.
26. Reviewer opens the timesheet.
27. Reviewer chooses Approve or Decline.
28. Server validates reviewer access and current state.
29. Review metadata is saved.
30. Employee receives a notification.
31. Activity history records the review event.

Review metadata

- Reviewer identity.
- Review timestamp.

- Decline reason when declined.

Important policy issue

- Supervisor eligibility is implemented differently in approval service logic and middleware. The product should define one explicit rule—for example, project supervisor OR direct supervisor, with admin override—and use it everywhere.

20. F-015 — Notifications

Purpose: inform users of events requiring awareness or action.

Observed notification scenarios

- Project/team-related events.
- Timesheet submission.
- Timesheet approval.
- Timesheet decline.
- Deadline reminders.
- Document-related events.

User features

- List current user's notifications.
- Display unread count.
- Mark one notification as read.
- Mark all notifications as read.

Automation

- A protected deadline endpoint can generate notifications for projects approaching their deadline.
- The current implementation should be made idempotent to prevent duplicate reminders when cron runs repeatedly.

Security requirement

- Mark-as-read must constrain the update by both notification ID and authenticated user ID. The current implementation does not enforce that ownership at the service call.

21. F-016 — Activity History & Audit Trail

Purpose: provide historical context for user, project and timesheet operations.

Activity dimensions

- User activity.
- Project activity.
- Timesheet activity.
- Timestamped descriptions.
- Associated user/project/timesheet identifiers.

Use cases

- Understand what happened to a project.
- Trace timesheet submission/review events.
- Display recent activity in dashboards.
- Support operational troubleshooting.

Audit caveat
The current implementation should not yet be treated as a tamper-proof compliance audit log. Activity creation is broadly exposed to authenticated clients, and caller-supplied user IDs are accepted. For a true audit trail, actor identity should always come from the authenticated session and sensitive events should be generated only by trusted services.

22. F-017 — Management Reporting

Purpose: give administrators aggregated visibility into workforce/project hours and timesheet state.

Report	Purpose	Audience
Hours by project	Aggregate hours associated with projects.	Admin
Hours by employee	Aggregate employee work hours.	Admin
Overtime	Compare regular and overtime totals.	Admin
Timesheet status	Break down draft/pending/approved/declined/withdrawn states.	Admin

Reporting considerations

- Reports are backed by MongoDB aggregation/service logic.
- Report access is admin-only.
- Date/project/employee filtering should be documented against the exact report service parameters before externalizing as a public reporting contract.

23. F-018 — Settings

Separate admin and user settings pages are present in the application.

- Settings are role-aware in navigation.
- The exact persisted configuration represented by every UI control should be verified before treating it as a supported backend capability.
- Security-sensitive settings should use backend validation and re-authentication where appropriate.

24. F-019 — Password Recovery

The API defines forgot-password and reset-password endpoints, but both explicitly return HTTP 501/not implemented.

Capability	Current state
Forgot password request	Not implemented
Reset password with token	Not implemented
Password email delivery	No implemented workflow identified
Reset-token persistence	No implemented workflow identified

This should be treated as a planned feature rather than a currently available user capability.

25. F-020 — Refresh Token Lifecycle

The backend contains refresh-token/session functionality, but there is no corresponding API route exposed for the frontend to refresh an expired access token.

- Access token expires after approximately one hour.
- Refresh token/session can remain valid for approximately seven days.
- Client currently retries a failed request once and can clear authentication on 401.
- A complete implementation should expose POST /auth/refresh and rotate refresh tokens.

Recommended product behavior
Users should remain signed in across the access-token lifetime as long as their refresh session is valid. The browser should transparently refresh an expired access token rather than forcing a logout.

26. F-021 — Deadline Notification Automation

Purpose: notify relevant users when project deadlines are approaching.

- Protected cron endpoint exists under notifications.
- The endpoint uses CRON_SECRET authorization.
- Projects with deadlines in the relevant window can trigger notifications.
- A production scheduler must invoke the endpoint.
- Notifications should be deduplicated/idempotent to avoid repeated alerts.

27. End-to-End Feature Flows

27.1 New employee → project → timesheet

```
Admin creates user
      ↓
Admin assigns supervisor (optional)
      ↓
Admin creates project
      ↓
Admin assigns manager/supervisor/team
      ↓
Employee sees accessible project
      ↓
Employee creates weekly timesheet
      ↓
Employee enters hours
      ↓
Employee submits
      ↓
Supervisor/Admin receives review item
      ↓
Approve / Decline
      ↓
Employee receives notification
      ↓
Activity history records outcome
```

27.2 Project document flow

```
Authorized project user
      ↓
Select file
      ↓
Multipart upload
      ↓
MIME + size validation
      ↓
Filename sanitization
      ↓
```

```
Vercel Blob upload
      ↓
MongoDB metadata
      ↓
Activity record
      ↓
Project document list / URL
```

27.3 Supervisor review flow

```
Supervisor login
      ↓
Supervisor role/capability verified
      ↓
Pending timesheets loaded
      ↓
Access scope applied
      ↓
Timesheet opened
      ↓
Approve or decline
      ↓
Review saved
      ↓
Notification + activity generated
```

28. Feature-Level Acceptance Criteria

Feature	Acceptance criteria
Authentication	Valid active user can log in; invalid credentials are rejected; protected endpoints reject unauthenticated requests; logout invalidates session.
Users	Admin can create/update/activate/deactivate users; duplicate email/employee ID is rejected; password hash is never returned.
Supervisors	Admin can assign/remove supervisor capability and assign supervisor relationships; supervisor screens appear only to eligible users.
Projects	Admin can create/update/delete projects; project has valid manager/team/supervisor relationships; authorized members can view it.
Documents	Authorized user can upload an allowed file ≤10 MB; metadata is stored; unauthorized users cannot access/delete another project's files.
Timesheets	User can create one timesheet per project/week; Monday normalization and hour rules are enforced; totals are server-calculated.
Submission	Editable timesheet can be submitted; pending timesheet enters review queue and triggers notification/activity.
Withdrawal	Owner can withdraw an eligible pending timesheet; resulting state is visible and editable according to policy.
Approval	Authorized reviewer can approve or decline; decline requires reason; review metadata and notifications are generated.
Notifications	User can list own notifications, read one own notification, and mark all own notifications read.
Activities	Authorized user can view permitted history; actor identity is server-derived for audit events.
Reports	Admin can access all four report families; non-admin is rejected.
Password recovery	If enabled, user can request and complete a secure reset workflow with expiring token.
Refresh	Valid refresh session produces a rotated access token without forcing user logout.

29. Edge Cases & Validation Matrix

Scenario	Expected behavior
Inactive user attempts login	Reject authentication.
Duplicate email	Reject with conflict/validation error.
Duplicate employee ID	Reject with conflict/validation error.
Duplicate project SOW number	Reject due to unique constraint.
Timesheet for non-member project	Reject.
Timesheet week not Monday	Normalize or reject according to service normalization.
Daily hours >24	Reject.
Hours without description	Reject.
Second timesheet for same user/project/week	Reject due to unique constraint.
Edit approved timesheet	Reject.
Approve non-pending timesheet	Reject.
Decline without reason	Reject.
Unauthorized project access	Reject with 403.
Unsupported document type	Reject upload.
Document >10 MB	Reject upload.
Mark another user's notification read	Must be rejected after security fix.
Activity attributed to another user	Must be rejected after security fix.
Expired access token with valid refresh	Should refresh after refresh flow is implemented.

30. UX Requirements Derived from the Current Feature Set

- Every destructive action should require confirmation where appropriate.
- Timesheet status should be visually obvious and consistently labeled.
- Approval screens should show project, employee, week, totals, notes, and review history before action.
- Declined timesheets should prominently expose the decline reason.
- Notifications should distinguish unread from read state.
- Project pages should make team, supervisor, dates, status and documents easy to find.
- Permission-denied states should explain what the user can do instead of presenting empty data.
- Loading and error states should be explicit for all asynchronous list and mutation operations.
- File upload UI should communicate accepted formats and the 10 MB limit.

31. Product Enhancement Roadmap

Release	Feature work	Rationale
R1 — Hardening	Fix notification ownership, activity attribution, document deletion policy, supervisor policy consistency.	Security and trust foundation.
R1 — Contracts	Align document upload/types and timesheet filters between frontend/backend.	Prevent broken UI/API interactions.
R1 — Authentication	Complete refresh endpoint and token rotation.	Reliable session continuity.
R2 — Collaboration	Improve project team/supervisor workflows, richer activity history, notification preferences.	Better operational collaboration.
R2 — Documents	Private/signed document access, previews, download controls, stronger lifecycle policy.	Safer document handling.
R2 — Reporting	Date ranges, project/employee filters, exports, saved report views.	Management usefulness.
R3 — Workflow	Configurable approval chains and escalation/reminder rules.	Support more organizations/processes.
R3 — Audit	Immutable audit event model, actor-derived events, retention policy.	Compliance/forensics.
R3 — Recovery	Full password reset and account recovery.	Operational completeness.

32. Feature Dependencies

```
Authentication
  └─> User Management
        ├─> Supervisor Management
        └─> Project Management
              ├─> Team Management
              ├─> Documents
              └─> Timesheets
                    └─> Submission
                          └─> Approval
                                ├─> Notifications
                                └─> Activities

Projects + Timesheets
  └─> Reports

Projects + Deadlines
  └─> Cron Notifications
```

33. Product-Level Risks

Risk	Severity	Effect
Authorization policy drift	High	Users may receive inconsistent access depending on endpoint path.
Public document URLs	High	Sensitive project files may be accessible to anyone with the URL.
Incomplete refresh flow	High	Users may be logged out after access-token expiry.
Notification ownership gap	High	Users may modify another user's notification state if an ID is known.
Activity attribution gap	High	Audit history can be misleading if clients can submit another actor's identity.
Frontend/backend DTO drift	High	Document UI/upload and other integrations may fail or display incorrect values.
Unpaginated list endpoints	Medium	Performance may degrade as users/projects/timesheets grow.
Non-transactional side effects	Medium	Primary state and notification/activity history can diverge after partial failures.
Demo/mock artifacts	Medium	Developers may accidentally rely on non-production data paths.

34. Final Feature Assessment

Eniac currently represents a substantial operational MVP rather than a simple CRUD application. Its central feature set—identity, projects, teams, weekly time capture, submission/review, notifications, activity tracking, documents and reporting—is present in the codebase and connected into meaningful workflows.

The most important product concern is consistency. Several features exist in one layer but differ in another. For a stable release, the platform should first make the implemented feature contracts explicit and canonical, especially around authorization, document handling, notifications, timesheet review and authentication renewal.

Recommended definition of 'production ready'
Do not define production readiness as simply 'all screens work.' For Eniac, production readiness should require: canonical authorization policies, secure resource ownership, aligned API/client contracts, completed authentication lifecycle, private or intentionally public document policy, automated security/integration tests, observable operations, and a documented backup/recovery process.